



Relatório do Projeto

Parte 1

Nome do Integrante	RA
Carolina Lee	10440304
Pedro Casas Pequeno Junior	10437031
Pedro Gabriel Guimarães Fernandes	10437465

Relatório

O relatório deve conter:

- 1) Título:
Code Recommendation

- 2) Integrantes (nome e e-mail com RA)
Carolina Lee
10440304
10440304@mackenzista.com.br

Pedro Casas Pequeno Junior
10437031
10437031@mackenzista.com.br

Pedro Gabriel Guimarães Fernandes
10437465
10437465@mackenzista.com.br

- 3) Resumo
Este projeto tem o objetivo de desenvolver um algoritmo que recomenda linguagens de programação para estudantes e profissionais de tecnologia e por meio de grafos, levando em conta os seus conhecimentos prévios, tipagem, paradigma entre outros.
- 4) Introdução
 - a. Contextualização

Um problema recorrente em estudantes de tecnologia é saber qual a próxima ou primeira linguagem que eles devem ver. Existem centenas de linguagens de programação, cada uma com características próprias como paradigmas, tipagem, aplicação e similaridade com outras linguagens. Tendo em vista que frequentemente alguns estudantes não sabem qual seria o próximo passo mais apropriado para tomar, esse projeto visa ser um apoiador na tomada de decisão sobre as diversas linguagens de programação, usando as similaridades entre elas como base para as recomendações.

- b. Justificativa

A modelagem por grafos é a abordagem mais adequada para este problema pois a relação entre as linguagens apresentam fatores que são compartilhados entre si, como paradigma com uma, tipagem com outra e aplicação com uma terceira,



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Teoria dos Grafos



formando uma rede de similaridades e não uma simples lista. Representar cada linguagem como um vértice, com atributos e arestas pelo grau de similaridade entre elas, permite usar algoritmos para identificar a partir do conjunto de linguagens já estudadas pelo usuário, quais seriam os próximos passos de maneira coerente com o perfil.

c. Objetivo

O objetivo principal é desenvolver o esquema de um algoritmo que recomenda linguagens de programação para o usuário estudar/aprofundar, levando em conta os seus conhecimentos prévios e em um grafo que modela as linguagens como vértices e suas semelhanças com as arestas. O esquema utilizará grafos para identificar as linguagens mais relevantes a serem recomendadas a partir do perfil atual do usuário.

5) Fundamentação Teórica (Resumida) ou Revisão da Literatura

a. Descrever os conceitos fundamentais, especialmente os relacionados ao uso da Teoria dos Grafos no projeto

Os vértices representam as diferentes linguagens de programação e as arestas são as relações entre linguagens e o peso nas arestas são a similaridade delas representada de forma quantificada.

Usaremos um grafo não direcionado com peso nas arestas.

Grafos não direcionados, são grafos em que suas arestas/relações não tem orientação, ou seja, é um grafo onde seus vértices estão conectados por arestas não-direcionadas.

A linguagem escolhida foi Java, visto que para o sistema de recomendação em grafos, será representado em programação orientada a objeto, que é mais fácil representar em java.

Para representar o grafo no código usaremos uma matriz de adjacência, por ser um grafo denso, ou seja, com uma quantidade alta de relações.

Usamos fórmulas de cosseno e jaccard para obter a similaridade, pegamos 2 artigos que usam elas para esse contexto, o Manning, Raghavan & Schütze (2008), cap. 6, pp. 117–121. para cosseno e Sheridan, Onsjö, Becerra, Jimenez & Dueñas (2019), cap. 2, pp. 4–6. para o Jaccard.

b. Trabalhos Relacionados: apresentar trabalhos similares ao do projeto proposto e a solução obtida por cada um deles.



Grafos – Aplicações

Projeto Disciplina 2025/1

Cinema Explorer – Sistema de Recomendação de Filmes – Projeto da Disciplina – 2025/1 – 6ºG – Alunos: Arthur Vignati Moscardi,

Davi Martins, Enzo Bernal, Ian Miranda da Cunha e Pedro Pessuto Rodrigues Ferreira.



Cada vértice armazena informações essenciais como ID, título, gêneros e avaliação média dos usuários. Conexões são estabelecidas entre filmes que compartilham um ou mais gêneros comuns. O problema central abordado é recomendar eficientemente filmes que estejam alinhados ao perfil do usuário, simplificando a complexidade das relações existentes.

Resumo: Este projeto visa desenvolver um sistema de recomendação de filmes utilizando estruturas de grafos para identificar e relacionar obras com características semelhantes. A principal motivação é facilitar a descoberta de filmes relevantes e ampliar o escopo de sugestões personalizadas aos usuários. Nesta abordagem, os filmes são representados como vértices, enquanto as relações entre eles, baseadas em gêneros compartilhados, são representadas como arestas.

Grafos - Aplicações

Booklizer - Um Aplicativo de Recomendação Literária Personalizada Usando GNN, Gabriella Silveira Braz, Giovana Liao, Maria Julia de Pádua (TCC - 2025/Artigo aprovado no ISLA 2025/ AW TIM – auxílio para Startup)

Resumo: este projeto propõe o desenvolvimento do aplicativo Booklizer que combina uma interface moderna com um modelo de recomendação baseado em GNN, capaz de captar relações semânticas entre livros por meio dos seus gêneros. A metodologia foi estruturada especialmente em: definição de requisitos; coleta e limpeza de uma base de dados literária, obtida do Kaggle; prototipação com Figma; implementação da interface com Flutter; e construção de um grafo bipartido entre livros e gêneros, sobre o qual foi treinada uma GNN. Os embeddings gerados foram avaliados por similaridade de cosseno, revelando agrupamentos temáticos coerentes e detecção de outliers



6) Metodologia:

- apresentar a abordagem que pretende empregar na resolução do problema, suas etapas.
 - Coleta de dados.
 - Modelagem do grafo.
 - Definição das linguagens de programação que utilizaremos como vértices.



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Teoria dos Grafos



- Definição das características que usaremos como fórmula para calcular similaridade.
- Definição da característica que utilizaremos para definir se há ou não uma aresta ligando dois vértices.
- Cálculo da similaridade entre linguagens.
- Busca da aresta com maior similaridade.

7) Resultados Parciais

a. Coleta dos dados: descrição detalhada

Definição das linguagens escolhidas

<https://www.bairesdev.com/blog/top-programming-languages/> -> 100 linguagens de programação mais usadas

<https://algorithmman.medium.com/top-100-list-of-programming-languages-e21c87c6075d> -> 100 linguagens de programação mais usadas,

Rosetta code,

A documentação de cada linguagem de programação.

b. Modelagem do problema para um Grafo: descrição detalhada;

As Linguagens de programação foram retiradas do <https://www.bairesdev.com/blog/top-programming-languages/>, dentre as 100 linguagens de programação mais usadas, retiramos as linguagens “mortas”, que não recebem mais atualizações de novas versões ou que as atualizações/novas versões demoram muito tempo para serem lançadas que são consideradas pela comunidade que a utiliza como uma linguagem morta.

As características usadas para detalhar cada linguagem são: paradigma da linguagem, tipagem forte ou fraca, estática ou dinâmica, se e qual linguagem de programação a influenciou, sintaxe e semântica da linguagem.

Para realizar o cálculo de similaridade foi utilizado um código em python com métodos de Jaccard e cosseno, retorna em um número de 0 a 1, sendo 0 nada similar e 1 muito similar, ligando todas linguagens entre si e suas respectivas similaridades.

O objetivo era utilizar as similaridades calculadas para modelar o grafo, todavia, o código retornou mais de 3 mil arestas, tornando inviável utilizar todas elas, por isso, definimos uma nota de corte para reduzirmos a quantidade de arestas, entendemos que o sistema de recomendação pode não ficar tão preciso por conta disso, mas precisávamos de um número de arestas mais fácil de operar/controlar. A nota de corte escolhida foi 0.2837, para todas as linguagens terem pelo menos uma aresta, já que, se tivéssemos um grafo/vértice isolado o sistema de recomendação não funcionaria como deveria.

c. Detalhamento dos resultados parciais obtidos até o momento;

Nossos resultados parciais são, um csv com os nós e suas similaridades com cada um deles (Observação: o arquivo resultante tem mais de 3 mil linhas, foi utilizado 700 delas para todos os vértices terem pelo menos uma ligação), um grafo estruturado no estilo de matriz de adjacência, um arquivo de saída em txt gerado pelo código, para poder analisar a matriz sem ser pelo terminal, e um grafo estruturado. Considerando o tamanho do nosso escopo, a plataforma que usamos para estruturar o nosso grafo foi a <https://keptune.ai/tools/ai-network-graph>, ela permite criar um grafo com um arquivo csv ou excel, mas testamos com a nossa saída em txt e funcional também.

Nosso grafo apresenta:



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

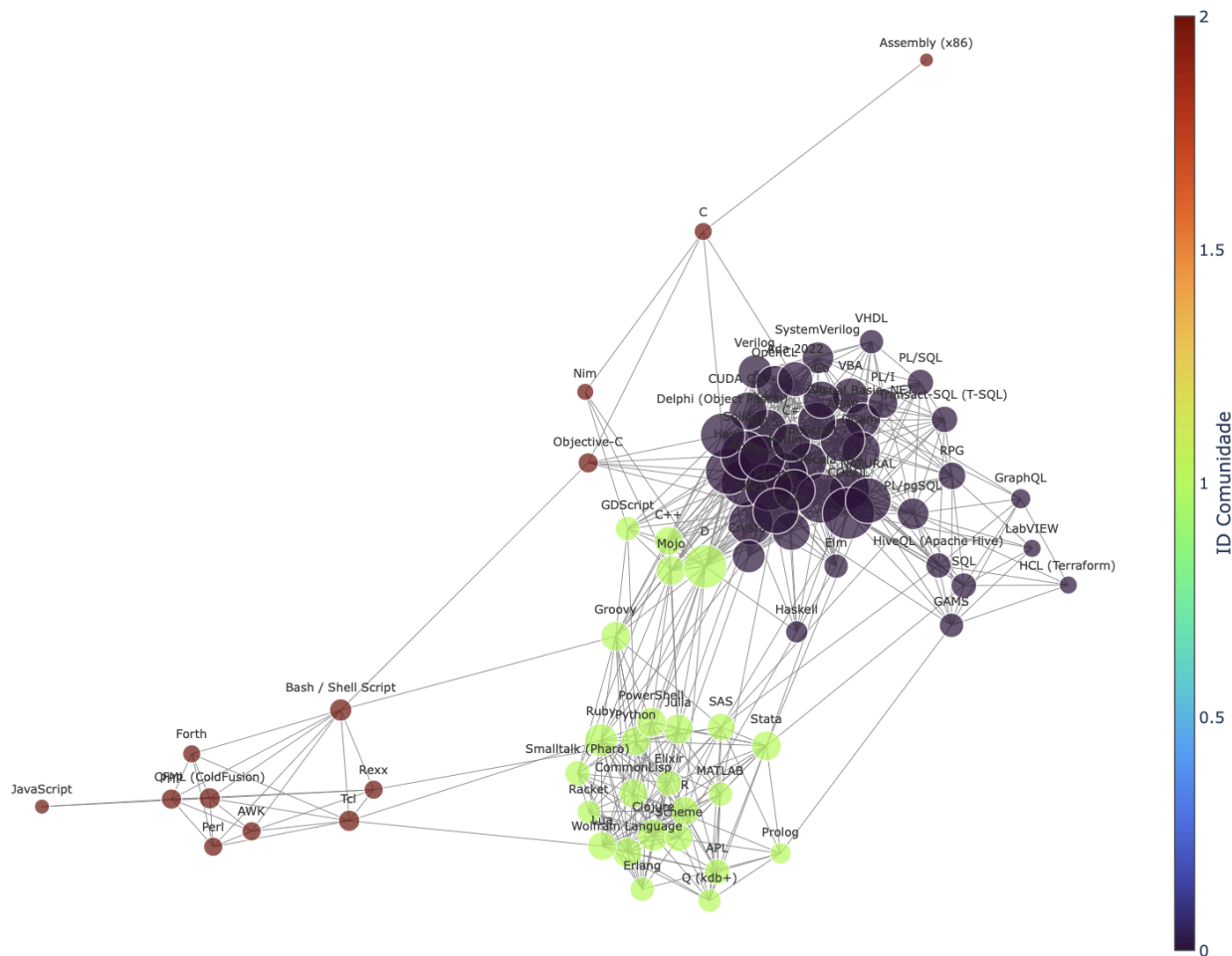
Prof. Dr. Ivan Carlos Alcântara de Oliveira

Teoria dos Grafos



81 Vértices, 700 arestas, conexo.

Grafo Não Direcionado com Pesos nas Arestas



Resultado CSV filtrado com as 700 linhas



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira
Teoria dos Grafos



```
[Versão Prévia] README.md lista_arestas_teste.csv X
Grafopasta > lista_arestas_teste.csv
1 no_a,no_b,peso
2 VBA,Visual Basic .NET,0.774
3 "PL/SQL","Transact-SQL, (T-SQL,)",0.657
4 F#,OCaml,0.6333
5 SAS,Stata,0.588
6 SystemVerilog,Verilog,0.5626
7 Dart,TypeScript,0.5623
8 "PL/pgSQL","Transact-SQL, (T-SQL,)",0.5577
9 GDScript,Mojo,0.5567
10 "CommonLisp","Racket,0.5557
11 Apex,Haxe,0.5539
12 Elm,Haskell,0.5451
13 Apex,X++,0.5266
14 "PL/pgSQL","PL/SQL,",0.5209
15 Dart,X++,0.5174
16 D,Dart,0.5067
17 D,Mojo,0.5031
18 Verilog,VHDL,0.5009
19 C++,D,0.4971
20 Haxe,Solidity,0.4969
21 Kotlin,Scala,0.495
22 D,Rust,0.4878
23 Ruby,Smalltalk (Pharo),0.4865
24 Apex,Dart,0.4847
25 CUDA C/C++,OpenCL,0.4831
26 Dart,Haxe,0.4779
27 HiveQL (Apache Hive),"PL/pgSQL,",0.4748
28 HiveQL (Apache Hive),"SQL,",0.4689
29 Crystal,F#,0.4656
30 MQL5,Solidity,0.4635
31 HCL (Terraform),"SQL,",0.4633
32 Dart,Solidity,0.4619
33 GAMS,"SQL,",0.4615
```



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira
Teoria dos Grafos



```
[Versão Prévia] README.md lista_arestas_teste.csv X
Grafopasta > lista_arestas_teste.csv
672 D,Haskell,0.2847
673 SystemVerilog,"Transact-SQL, (T-SQL,)",0.2846
674 CUDA C/C++,OCaml,0.2846
675 Crystal,Solidity,0.2846
676 D,Ruby,0.2845
677 Apex,"PL/pgSQL,",0.2845
678 Go,Solidity,0.2844
679 F#,PL/I,0.2844
680 Apex,NATURAL,0.2843
681 CUDA C/C++,Kotlin,0.2843
682 Groovy,Wolfram Language,0.2843
683 APL,SAS,0.2842
684 Go,NATURAL,0.2842
685 Solidity,VBA,0.2841
686 OCaml,Solidity,0.2841
687 Solidity,Visual Basic .NET,0.2841
688 Ada 2022,Scala,0.2841
689 APL,Python,0.284
690 "Fortran,",Go,0.284
691 "Fortran,",Scala,0.284
692 Julia,Racket,0.284
693 AWK,Rexx,0.2839
694 VBA,Verilog,0.2839
695 NATURAL,Visual Basic .NET,0.2839
696 NATURAL,VBA,0.2839
697 Verilog,Visual Basic .NET,0.2839
698 Kotlin,NATURAL,0.2837
699 Java,PL/I,0.2837
700 Lua,Tcl,0.2837
701 GraphQL,Kotlin,0.2837
```

Resultados dos códigos (a saída é bem longa, então pegamos fragmentos que mostram o principal dela) *o resultado no txt é o mesmo do terminal:



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira
Teoria dos Grafos



```
Problemas Saída Console de Depuração Terminal Portas
@PedroCGithub →/workspaces/CodeRecomendation (main) $ /usr/bin/env /usr/local/sdkman/candidates/java/25.0.4+1-ms
ages -cp /home/codespace/.vscode-remote/data/User/workspaceStorage/-23dc5c4a/redhat.java/jdt_ws/CodeRecomendation
Vertices lidos: 82
Arestas lidas: 700

Mapeamento de vertices:
0 = VBA
1 = Visual Basic .NET
2 = PL/SQL,
3 = Transact-SQL, (T-SQL,)
4 = F#
5 = OCaml
6 = SAS
7 = Stata
8 = SystemVerilog
9 = Verilog
10 = Dart
11 = TypeScript
12 = PL/pgSQL,
13 = GDScript
14 = Mojo
15 = CommonLisp,
16 = Racket
17 = Apex
18 = Haxe
19 = Elm
20 = Haskell
21 = X++
22 = D
23 = VHDL

@PedroCGithub →/workspaces/CodeRecomendation (main) $ /usr/bin/env /usr/local/sdkman/candidates/java/25.0.4+1-ms
ages -cp /home/codespace/.vscode-remote/data/User/workspaceStorage/-23dc5c4a/redhat.java/jdt_ws/CodeRecomendation
72 = Ada 2022
73 = Lua
74 = Nim
75 = PowerShell
76 = JavaScript
77 = LabVIEW
78 = C
79 = Forth
80 = Rexx
81 = Assembly (x86)
n: 82
m: 700

Adj[0,0]=inf Adj[0,1]=0.774 Adj[0,2]=0.3161 Adj[0,3]=0.3144 Adj[0,4]=inf Adj[0,5]=inf Adj[0,6]=inf Adj[0,7]=inf Adj[0,8]=inf Adj[0,9]=inf Adj[0,10]=inf Adj[0,11]=0.2917 Adj[0,12]=inf Adj[0,13]=inf Adj[0,14]=inf Adj[0,15]=inf Adj[0,16]=inf Adj[0,17]=0.3172 Adj[0,18]=0.3277 Adj[0,19]=inf Adj[0,20]=inf Adj[0,21]=inf Adj[0,22]=inf Adj[0,23]=inf Adj[0,24]=inf Adj[0,25]=0.2841 Adj[0,26]=0.3074 Adj[0,27]=0.306 Adj[0,28]=inf Adj[0,29]=inf Adj[0,30]=inf Adj[0,31]=inf Adj[0,32]=0.2973 Adj[0,33]=inf Adj[0,34]=inf Adj[0,35]=inf Adj[0,36]=0.2851 Adj[0,37]=inf Adj[0,38]=inf Adj[0,39]=inf Adj[0,40]=inf Adj[0,41]=inf Adj[0,42]=inf Adj[0,43]=0.2864 Adj[0,44]=inf Adj[0,45]=inf Adj[0,46]=inf Adj[0,47]=inf Adj[0,48]=0.3168 Adj[0,49]=0.2955 Adj[0,50]=inf Adj[0,51]=inf Adj[0,52]=inf Adj[0,53]=inf Adj[0,54]=inf Adj[0,55]=inf Adj[0,56]=inf Adj[0,57]=0.2942 Adj[0,58]=0.2917 Adj[0,59]=inf Adj[0,60]=inf Adj[0,61]=inf Adj[0,62]=inf Adj[0,63]=inf Adj[0,64]=inf Adj[0,65]=inf Adj[0,66]=inf Adj[0,67]=inf Adj[0,68]=inf Adj[0,69]=inf Adj[0,70]=inf Adj[0,71]=inf Adj[0,72]=inf Adj[0,73]=inf Adj[0,74]=inf Adj[0,75]=inf Adj[0,76]=inf Adj[0,77]=inf Adj[0,78]=inf Adj[0,79]=inf Adj[0,80]=inf Adj[0,81]=inf Adj[1,0]=0.774 Adj[1,1]=inf Adj[1,2]=0.2917 Adj[1,3]=0.2917 Adj[1,4]=inf Adj[1,5]=inf Adj[1,6]=inf Adj[1,7]=inf Adj[1,8]=inf Adj[1,9]=inf Adj[1,10]=inf Adj[1,11]=0.2917 Adj[1,12]=inf Adj[1,13]=inf Adj[1,14]=inf Adj[1,15]=inf Adj[1,16]=inf Adj[1,17]=0.2947 Adj[1,18]=0.2973 Adj[1,19]=inf Adj[1,20]=inf Adj[1,21]=inf Adj[1,22]=inf Adj[1,23]=inf Adj[1,24]=inf Adj[1,25]=0.2841 Adj[1,26]=0.34 Adj[1,27]=0.3235 Adj[1,28]=inf Adj[1,29]=inf Adj[1,30]=inf Adj[1,31]=inf Adj[1,32]=inf Adj[1,33]=inf Adj[1,34]=inf Adj[1,35]=inf Adj[1,36]=0.2851 Adj[1,37]=inf Adj[1,38]=inf Adj[1,39]=inf Adj[1,40]=inf Adj[1,41]=inf Adj[1,42]=inf Adj[1,43]=inf Adj[1,44]=inf Adj[1,45]=inf Adj[1,46]=inf Adj[1,47]=inf Adj[1,48]=0.3168 Adj[1,49]=0.341 Adj[1,50]=inf Adj[1,51]=inf Adj[1,52]=inf Adj[1,53]=inf Adj[1,54]=inf Adj[1,55]=inf Adj[1,56]=inf Adj[1,57]=inf Adj[1,58]=inf Adj[1,59]=inf Adj[1,60]=inf Adj[1,61]=inf Adj[1,62]=inf Adj[1,63]=inf Adj[1,64]=inf Adj[1,65]=inf Adj[1,66]=inf Adj[1,67]=inf Adj[1,68]=inf Adj[1,69]=inf Adj[1,70]=inf Adj[1,71]=inf Adj[1,72]=inf Adj[1,73]=inf Adj[1,74]=inf Adj[1,75]=inf Adj[1,76]=inf Adj[1,77]=inf Adj[1,78]=inf Adj[1,79]=inf Adj[1,80]=inf Adj[1,81]=inf
```



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Teoria dos Grafos



```
81,11]=inf Adj[81,12]=inf Adj[81,13]=inf Adj[81,14]=inf Adj[81,15]=inf Adj[81,16]=inf Adj[81,17]=inf Adj[81,18]=inf Adj[81,19]=inf Adj[81,20]=inf Adj[81,21]=inf Adj[81,22]=inf Adj[81,23]=inf Adj[81,24]=inf Adj[81,25]=inf Adj[81,26]=inf Adj[81,27]=inf Adj[81,28]=inf Adj[81,29]=inf Adj[81,30]=inf Adj[81,31]=inf Adj[81,32]=inf Adj[81,33]=inf Adj[81,34]=inf Adj[81,35]=inf Adj[81,36]=inf Adj[81,37]=inf Adj[81,38]=inf Adj[81,39]=inf Adj[81,40]=inf Adj[81,41]=inf Adj[81,42]=inf Adj[81,43]=inf Adj[81,44]=inf Adj[81,45]=inf Adj[81,46]=inf Adj[81,47]=inf Adj[81,48]=inf Adj[81,49]=inf Adj[81,50]=inf Adj[81,51]=inf Adj[81,52]=inf Adj[81,53]=inf Adj[81,54]=inf Adj[81,55]=inf Adj[81,56]=inf Adj[81,57]=inf Adj[81,58]=inf Adj[81,59]=inf Adj[81,60]=inf Adj[81,61]=inf Adj[81,62]=inf Adj[81,63]=inf Adj[81,64]=inf Adj[81,65]=inf Adj[81,66]=inf Adj[81,67]=inf Adj[81,68]=inf Adj[81,69]=inf Adj[81,70]=inf Adj[81,71]=inf Adj[81,72]=inf Adj[81,73]=inf Adj[81,74]=inf Adj[81,75]=inf Adj[81,76]=inf Adj[81,77]=inf Adj[81,78]=0.2896 Adj[81,79]=inf Adj[81,80]=inf Adj[81,81]=inf

fim da impressao do grafo.

Resultado salvo em: Grafopasta/grafos_resultado.txt
@PedroCGithub -> /workspaces/CodeRecommendation (main) $
```

Teste da remoção de vértice e aresta
teste rodado:

```
System.setOut(arquivo);

mostraVertices(vertices);
grafo.removeAND(v: 0, w: 1);
grafo.removeVND(v: 2);
grafo.showND();

arquivo.close();
```

Resultado obtido, o número de vértice caiu e suas arestas também foram removidas, e houve a remoção de uma das arestas do vértice 0 com o 1

Se comparar com o print anterior, verá que o valor de [0, 1] foi de 0,774 para inf, mostrando a exclusão dele. Também podemos ver que o valor de [0, 2] se tornou 0,3144, que era o valor de [0, 3] mostrando que o vértice 2 foi excluído e a matriz modificou o que precisava



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira
Teoria dos Grafos



```
[Versão Prévia] README.md  J TesteGrafoNDPeso.java M  grafo_resultado.txt M X
Grafopasta > grafo_resultado.txt
75  72 = Ada 2022
76  73 = Lua
77  74 = Nim
78  75 = PowerShell
79  76 = JavaScript
80  77 = LabVIEW
81  78 = C
82  79 = Forth
83  80 = REXX
84  81 = Assembly (x86)
85  Vértice 2 removido com sucesso.
86  n: 81
87  m: 686
88
89  Adj[0,0]=inf Adj[0,1]=inf Adj[0,2]=0.3144 Adj[0,3]=inf Adj[0,4]=inf Adj
90  Adj[1,0]=inf Adj[1,1]=inf Adj[1,2]=0.2917 Adj[1,3]=inf Adj[1,4]=inf Adj
91  Adj[2,0]=0.3144 Adj[2,1]=0.2917 Adj[2,2]=inf Adj[2,3]=0.2963 Adj[2,4]=0
92  Adj[3,0]=inf Adj[3,1]=inf Adj[3,2]=0.2963 Adj[3,3]=inf Adj[3,4]=0.6333
```

d. Endereço GitHub criado contendo o descrito a seguir.

<https://github.com/roxo-1/CodeRecomendation>

8) Referências: obtidas por pesquisas e citadas dentro do texto do projeto

Links usados para pesquisas:

<https://www.bairesdev.com/blog/top-programming-languages/>

<https://algorithmman.medium.com/top-100-list-of-programming-languages-e21c87c6075d>

https://rosettacode.org/wiki/Rosetta_Code

A documentação de cada linguagem de programação. Aqui são muitos links, temos uma planilha com eles que usamos para as informações, aqui iria ficar muito poluído se colocasse tudo.

<https://keptune.ai/tools/ai-network-graph>

<https://www.dio.me/articles/jaccard-vs-cosine-similarity-entendendo-as-diferencas-na-analise-de-grafos-469bb6abb9b2>

Artigos usados:

Cosseno: Manning, Raghavan & Schütze (2008), cap. 6, pp. 117–121.

<https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

Jaccard: Sheridan, Onsjö, Becerra, Jimenez & Dueñas (2019), cap. 2, pp. 4–6.

<https://arxiv.org/pdf/1808.00103>



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Teoria dos Grafos

SHERIDAN, P.; ONSJÖ, M.; BECERRA, C.; JIMENEZ, S.; DUEÑAS, G. An Ontology-Based Recommender System with an Application to the Star Trek Television Franchise. *arXiv preprint* arXiv:1808.0



9) Planejamento futuro

Planejamos futuramente montar as telas iniciais usando alguma plataforma de design como figma, porém, devido a escala do projeto e tempo dos integrantes, o funcionamento do programa em si vamos manter por terminal, tendo opções para o usuário escolher, podendo fornecer as informações que precisamos para fazer a recomendação.

A área do GitHub deste bimestre, com acesso público, deve apresentar:

- Relatório do Projeto; <https://github.com/roxo-1/CodeRecomendation>
- Os arquivos fontes do conteúdo desenvolvido até o momento, que devem contemplar:
 - um cabeçalho (comentário) com as identificações completas de todos os membros do grupo, síntese do conteúdo do arquivo e histórico das alterações realizadas, no formato: data da atualização, autor, breve descrição da atualização.